

Materia: Programación II – 3° Trabajo Práctico.

Profesor: Ing. Tulio Ruesjas Martín.

Nombre y Apellido del Alumno: Montoya Gonzalo Alejo – CI180156

El presente trabajo práctico tiene 5 ejercicios hechos en Java y se centra en herencia, clases abstractas e interfaces. A diferencia de los TPs anteriores, acá ya no alcanza con escribir todo en una sola clase, sino que hay que pensar bien cómo organizar el código en jerarquías. Al principio me costó entender cuándo usar una clase abstracta y cuándo una interfaz, pero con los ejercicios lo fui entendiendo mejor.

Ejercicio 1 *El ejercicio pedía modelar una flota de vehículos con furgonetas y motocicletas. Se creó la clase abstracta Vehiculo con los atributos comunes (marca, modelo y tarifa base) y el método abstracto mostrarInformacion(). Cada subclase hereda lo común y agrega su propio atributo: la furgoneta tiene capacidad de carga y la moto tiene cilindrada. El polimorfismo se ve en el Main cuando se recorre la lista y cada vehículo imprime su propia información según su tipo.*

Ejercicio 2 *Se modeló un sistema de suscripciones de streaming con tres planes distintos. La clase abstracta Suscripcion agrupa el email, número de cliente y costo base, pero no puede calcular el costo real porque no sabe de qué plan se trata, por eso el método calcularCostoMensual() es abstracto. El PlanBasico cobra solo la base, el PlanFamiliar suma un recargo por cada perfil adicional y el PlanPremium agrega un cargo fijo por 4K y descargas. El motor de facturación recorre la lista y llama al método en cada suscripción sin importar el tipo.*

Ejercicio 3 *Este ejercicio modela una pasarela de pagos para una tienda virtual que acepta tarjeta de crédito, PayPal y criptomonedas. Como las tres plataformas no tienen nada en común más allá de procesar un monto, se usó una interfaz en lugar de una clase abstracta. La interfaz PasarelaPago define solo el método procesarPago(double monto) y cada clase lo implementa con su propia lógica. El carrito solo conoce la interfaz y no le importa qué hay detrás, lo que permite agregar nuevas formas de pago sin tocar el resto del código.*

Ejercicio 4 *Se modeló un ecosistema de dispositivos inteligentes donde algunos pueden tomar fotos, otros conectarse a WiFi, y el smartphone puede hacer ambas cosas. Acá es donde se entiende bien por qué existen las interfaces: Java no permite heredar de dos clases, pero sí implementar múltiples interfaces. Se crearon las interfaces Fotografiable y ConectableWifi por separado, y el Smartphone implementa las dos. El controlador usa instanceof para verificar qué capacidad tiene cada dispositivo antes de ejecutar la acción.*

Ejercicio 5 *Es el más complejo del TP. Se diseñó un motor de videojuego de rol con temática estilo Diablo, con una jerarquía de cuatro niveles: EntidadEspacial → SerVivo → PersonajeJugable / MonstruoHostil → clases concretas. Adicionalmente existe la interfaz LanzadorDeHechizos que cruza la jerarquía: la Hechicera (personaje) y Diablo (monstruo) pueden lanzar hechizos, pero el Bárbaro y el Esqueleto no. El motor puede mover todo, calcular daño con reducción por defensa, y lanzar ataques mágicos solo a quienes dominan la magia.*

Lo que más me costó fue entender bien la diferencia entre clase abstracta e interfaz. Al principio usaba abstracta para todo, pero me di cuenta que la interfaz tiene sentido cuando querés definir un comportamiento que puede tener cualquier clase, sin importar de dónde venga. También tuve que pensar bien el orden de los constructores con super() porque si no lo ponés primero no compila. En general este TP fue bastante más difícil que los anteriores pero también más interesante porque el código empieza a parecerse a algo real.